EE627A Midterm

Heuristic-Based Music Recommendation

Jude Eschete

Course: EE627A — Data Acquisition and Processing I: Big Data

Instructor: Prof. Rensheng Wang

Semester: Spring 2026

Date: March 31, 2026

Contents

1 Part 1: Heuristic-Based Music Recommendation **1**

1.1 Objective . . . 1

1.2 Dataset Overview . . . 1

1.3 Part 1.a: Feature Engineering & Statistical Aggregation . . . 2

1.3.1 Methodology . . . 2

1.3.2 Data Sparsity Analysis . . . 2

1.3.3 Feature Vector Example – User 199810 . . . 3

1.4 Part 1.b: Decision Logic & Rule Definition . . . 4

1.4.1 Strategy 1: Maximum Genre Score . . . 4

1.4.2 Strategy 2: Weighted Average . . . 4

1.4.3 Strategy 3: Evidence-Weighted Composite . . . 4

1.4.4 Cross-Strategy Agreement . . . 5

1.5 Python Implementation . . . 5

1.6 Part 1 Results . . . 6

1.6.1 Submission Output . . . 6

1.6.2 Kaggle Leaderboard Results . . . 6

1.6.3 Part 1 Key Observations . . . 6

2 Part 2: Addressing the Cold Start Problem **8**

2.1 Objective . . . 8

2.2 Part 2.a: The Global Fallback . . . 8

2.2.1 Strategy Design . . . 8

2.3 Part 2.b: Dig Deeper Search Logic . . . 9

2.3.1 Strategy Design . . . 9

2.3.2 Hierarchical Fallback Implementation . . . 9

2.4 Part 2 Python Implementation . . . 9

2.5 Part 2 Impact Analysis . . . 10

2.5.1 Fallback Source Distribution . . . 10

2.5.2 Prediction Changes . . . 10

2.5.3 Kaggle Leaderboard Results . . . 10

2.5.4 Key Findings — Why Fallback Scores Decreased . . . 11

3 All Submission Files **13**

A Python Source Code **14**

A.1 Data Parsing . . . 14

A.2 Feature Engineering (Part 1) . . . 16

A.3 Heuristic Strategies (Part 1) . . . 17

A.4 Ranking and Submission Pipeline 18
A.5 Main Entry Point (Part 1) 19
A.6 Part 2: Cold-Start Extensions 20

List of Figures

1 Kaggle AUC-ROC scores for the three Part 1 strategies. 6
2 Kaggle AUC-ROC scores for the Part 2 strategies. 10

List of Tables

1 Dataset statistics. 1
2 Feature coverage across all 120,000 test candidates. 2
3 Feature vectors for the first test user (User 199810). 3
4 Pairwise agreement between strategies (out of 120,000 predictions). 5
5 Part 1 Kaggle AUC-ROC comparison. 6
6 Album score source breakdown with Global Fallback. 8
7 Album score source: Global-only vs. full Hierarchical fallback. 10
8 Prediction changes vs. Part 1 baseline (WeightedAvg strategy). 10
9 Complete Kaggle AUC-ROC comparison: Part 1 vs. Part 2. 11
10 Complete list of Kaggle submission files. 13

List of Equations

1 Strategy 1: Maximum Genre Score 4
2 Strategy 2: Weighted Average 4
3 Strategy 3: Evidence-Weighted Composite 4
4 Global Popularity Score 8
5 Album Proxy via Sibling Tracks (Dig Deeper) 9

1 Part 1: Heuristic-Based Music Recommendation

1.1 Objective

The goal is to develop a rule-based recommendation system for the Yahoo Music dataset. For each of 20,000 test users, the system is given six candidate tracks and must label exactly three as "1" (Recommend) and three as "0" (Do Not Recommend). The system is evaluated by AUC-ROC on the Kaggle leaderboard.

1.2 Dataset Overview

The Yahoo Music dataset stores music data in a hierarchical structure:

$$\textbf{Track} \rightarrow \textbf{Album} \rightarrow \textbf{Artist} \rightarrow \textbf{Genre(s)}$$

Users rate items across all hierarchy levels on a scale of 0–100. The training set contains ratings for tracks, albums, artists, and genres in a single flat ID space.

Property	**Value**
Training users	49,204
Total ratings	12,403,575
Tracks in metadata	224,041
Albums in metadata	52,829
Test users	20,000
Candidates per user	6
Total test candidates	120,000

Table 1: Dataset statistics.

1.3 Part 1.a: Feature Engineering & Statistical Aggregation

1.3.1 Methodology

For each candidate track, the system looks up the track's hierarchy (Album ID, Artist ID, and Genre IDs) from `trackData2.txt`, then searches the test user's training history for direct ratings of those hierarchy items.

The resulting feature vector for each (user, candidate-track) pair contains:

- **Album Score**: The user's historical rating for the track's album (0 if not rated).
- **Artist Score**: The user's historical rating for the track's artist (0 if not rated).
- **Genre Statistics**: Computed over all of the track's genres that the user has rated:
  - **Count**: Number of the track's genres rated by the user.
  - **Maximum**: Highest rating among matched genres.
  - **Minimum**: Lowest rating among matched genres.
  - **Mean**: Average rating across matched genres.
  - **Variance**: Variance of ratings across matched genres.
  - **Sum**: Total of ratings across matched genres.
- **Has Album / Has Artist**: Binary flags indicating whether the user has rated the album or artist at all.

1.3.2 Data Sparsity Analysis

A critical observation is that the training data is sparse at the hierarchy level. Not every user has rated the album, artist, or genres of every candidate track.

Feature	**Count Present**	**Coverage (%)**
Album rating	34,264	28.6%
Artist rating	51,904	43.3%
$\geq$1 Genre rating	58,223	48.5%
Avg. genre matches per candidate	0.70	

Table 2: Feature coverage across all 120,000 test candidates.

This sparsity directly influences the strategy design: any heuristic that relies solely on album scores will have no information for 71.4% of candidates. Artist and genre signals provide broader coverage and are essential for discriminating among candidates.

1.3.3 Feature Vector Example – User 199810

TrackID	AlbumID	ArtistID	AlbScr	ArtScr	G_Cnt	G_Max	G_Min	G_Mean	G_Var
208019	209288	None	0	0	0	0	0	0.0	0.0
74139	277282	271146	0	0	1	80	80	80.0	0.0
9903	None	None	0	0	0	0	0	0.0	0.0
242681	190640	244574	0	0	0	0	0	0.0	0.0
18515	146344	33168	0	70	0	0	0	0.0	0.0
105760	93458	11616	0	90	2	80	80	80.0	0.0

Table 3: Feature vectors for the first test user (User 199810).

For User 199810, only three of six candidates have any training signal. Tracks 74139, 18515, and 105760 have artist and/or genre matches, while the remaining three tracks have no historical evidence. All three strategies correctly recommend the three tracks with non-zero feature values.

1.4 Part 1.b: Decision Logic & Rule Definition

Three heuristic strategies were implemented, each defining a composite score to rank the six candidates. The top three by score are recommended.

1.4.1 Strategy 1: Maximum Genre Score

$$\text{Score} = 0.30 \times \text{AlbumScore} + 0.20 \times \text{ArtistScore} + 0.50 \times \text{GenreMax} \tag{1}$$

Rationale: Genre preference is a fundamental taste indicator. If a user gives a high rating to *any* genre associated with a track, the track is likely to resonate with that user. The maximum (rather than mean) captures the "best-case" genre alignment and avoids dilution from low-rated secondary genres. Album and artist scores provide additional specificity.

1.4.2 Strategy 2: Weighted Average

$$\text{Score} = 0.40 \times \text{AlbumScore} + 0.35 \times \text{ArtistScore} + 0.25 \times \text{GenreMean} \tag{2}$$

Rationale: This strategy balances all three hierarchy levels with the strongest weight on the album (the most granular personalized signal, since the same album implies high content similarity), followed by artist affinity, and complemented by the user's average genre preference across all of the track's genres.

1.4.3 Strategy 3: Evidence-Weighted Composite

$$\text{Score} = \text{HasAlbum} \times \text{AlbumScore} + \text{HasArtist} \times \text{ArtistScore} + \frac{\text{GenreCount} \times \text{GenreMean}}{10} \tag{3}$$

Rationale: This strategy explicitly addresses data sparsity. A zero album score for an un-rated album is absence-of-evidence, not evidence-of-absence. The binary flags (`HasAlbum`, `HasArtist`) ensure that only *actually rated* hierarchy items contribute to the score. Genre breadth (count $\times$ mean) rewards tracks where the user has interacted with many of the track's genres, providing stronger cumulative evidence.

1.4.4 Cross-Strategy Agreement

Comparison	Agreement	Rate
MaxGenre vs. WeightedAvg	113,428	94.5%
MaxGenre vs. Evidence	111,722	93.1%
WeightedAvg vs. Evidence	117,138	97.6%

Table 4: Pairwise agreement between strategies (out of 120,000 predictions).

The strategies agree on 93–98% of predictions, indicating that the underlying signals (album, artist, genre) are consistent. The remaining divergence comes from edge cases where sparsity handling or genre aggregation (max vs. mean) shifts the ranking of borderline candidates.

1.5 Python Implementation

The complete Python implementation for Part 1 is presented in Appendix A. The code handles data parsing, feature engineering, the three heuristic strategies, ranking, and Kaggle submission generation.

1.6 Part 1 Results

1.6.1 Submission Output

Each strategy produces a Kaggle-format CSV with 120,000 rows (20,000 users $\times$ 6 candidates). Exactly 60,000 predictions are labeled "1" (recommend) and 60,000 labeled "0" (do not recommend), satisfying the constraint of 3 recommendations per user.

1.6.2 Kaggle Leaderboard Results

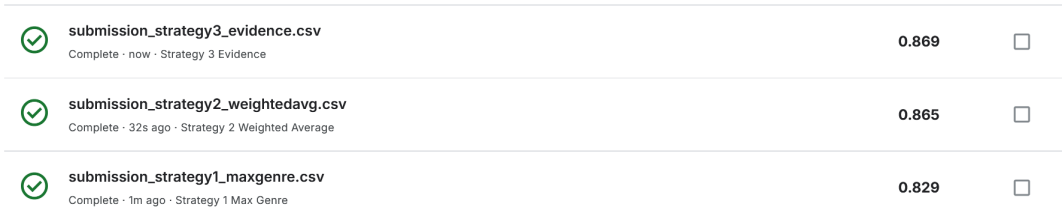

Figure 1: Kaggle AUC-ROC scores for the three Part 1 strategies.

Strategy	Scoring Rule	AUC-ROC
S1: Max Genre Score	$0.30 \cdot \text{album} + 0.20 \cdot \text{artist} + 0.50 \cdot \text{genre_max}$	0.829
S2: Weighted Average	$0.40 \cdot \text{album} + 0.35 \cdot \text{artist} + 0.25 \cdot \text{genre_mean}$	0.865
S3: Evidence-Weighted	$\text{has_alb} \cdot \text{album} + \text{has_art} \cdot \text{artist} + \frac{\text{g_cnt} \cdot \text{g_mean}}{10}$	**0.869**

Table 5: Part 1 Kaggle AUC-ROC comparison.

1.6.3 Part 1 Key Observations

1. **Evidence-Weighted wins (AUC = 0.869):** Strategy 3 outperforms the other two by explicitly addressing data sparsity. Rather than assigning a zero to un-rated albums/artists (which drags down scores for tracks the user may actually like), it only counts an item when the user has *actually* rated it. This avoids penalizing tracks with missing data.

2. **Weighted Average is a close second (AUC = 0.865):** Balancing album, artist, and mean genre scores is effective. The small gap to S3 suggests that the fixed zero-imputation for missing scores is only mildly harmful when all three hierarchy levels are combined.

3. **Max Genre Score underperforms (AUC = 0.829):** Placing 50% weight on a single genre's maximum rating is too aggressive. In a sparse dataset where only 48.5% of candidates have any genre match, the genre_max signal is zero for half the candidates, reducing discriminative power.

4. **Severe data sparsity:** Only 28.6% of candidates have a direct album rating, 43.3% have an artist rating, and 48.5% have at least one genre match. For roughly half of all candidates, the system has zero personalized features—motivating the cold-start strategies in Part 2.

2 Part 2: Addressing the Cold Start Problem

2.1 Objective

Part 1's data sparsity analysis revealed that 71.4% of candidates lack a direct album rating, 56.7% lack an artist rating, and 51.5% have no genre match at all. Part 2 develops fallback strategies for these "cold start" scenarios.

2.2 Part 2.a: The Global Fallback

2.2.1 Strategy Design

For candidates where the user has no direct rating for the album, artist, or genre, the system falls back to **global popularity**—the average rating for that item across *all* users in the training set.

Global popularity computation: For each item ID in the training set, compute:

$$\text{GlobalScore}(item) = \frac{\sum_{u \in \text{raters}(item)} r_{u,item}}{|\text{raters}(item)|} \tag{4}$$

This produces a global mean rating and rater count for every track, album, artist, and genre (295,799 items total).

Feature imputation: When a user has not rated a candidate's album (or artist, or genre), the system substitutes the global average rating for that item. This provides a non-personalized "universally liked" signal that is better than the zero-score default from Part 1.

Source	Count	Percentage
Direct (user rated album)	34,264	28.6%
Global (avg. across users)	77,164	64.3%
None (album ID unknown)	8,572	7.1%

Table 6: Album score source breakdown with Global Fallback.

Global imputation fills album scores for 64.3% of candidates that had no direct rating. The remaining 7.1% have an album ID of `None` in the track metadata, so no imputation is possible.

2.3 Part 2.b: Dig Deeper Search Logic

2.3.1 Strategy Design

When a user has no direct rating for a candidate track's album, the system performs a "Dig Deeper" search: it looks for **sibling tracks**—other tracks that belong to the same album—that the user *has* rated.

The logic: Given a candidate track t with Album ID a:

1. Build an index: album $\rightarrow$ [track IDs] from `trackData2.txt`
2. If the user has not directly rated album a, find all tracks $\{t_1, t_2, \ldots\}$ that share album a
3. Check which of those sibling tracks the user has rated in training
4. If any exist, compute the proxy album score as:

$$\text{AlbumProxy}(u, a) = \frac{1}{|S|} \sum_{t_i \in S} r_{u,t_i} \quad \text{where } S = \{t_i \in \text{album}(a) \mid t_i \in \text{rated}(u)\} \tag{5}$$

The mean aggregation is chosen over the maximum because it provides a more robust estimate—a single high-rated sibling could be an outlier, while the average reflects the user's overall engagement with the album's content.

2.3.2 Hierarchical Fallback Implementation

The full pipeline follows this priority:

1. **Primary:** Direct Album/Artist/Genre scores from user history
2. **Secondary (Dig Deeper):** If album score is missing, compute mean sibling-track rating for that album
3. **Tertiary (Global):** If no sibling tracks found, use the global average rating

Artist and genre scores use a simpler two-level fallback (Direct $\rightarrow$ Global).

2.4 Part 2 Python Implementation

The Part 2 Python code—including the album-to-tracks index, global statistics computation, and the enhanced feature vector with hierarchical fallback—is presented in Appendix A.

2.5 Part 2 Impact Analysis

2.5.1 Fallback Source Distribution

Source	Global Only		Hierarchical	
	Count	%	Count	%
Direct	34,264	28.6	34,264	28.6
Dig Deeper	0	0.0	5,678	4.7
Global	77,164	64.3	71,486	59.6
None	8,572	7.1	8,572	7.1

Table 7: Album score source: Global-only vs. full Hierarchical fallback.

The Dig Deeper logic identified 5,678 candidates (4.7%) where the user had not rated the album directly but *had* rated sibling tracks in the same album. These candidates shifted from the less-personalized "Global" source to the more-personalized "Dig Deeper" proxy.

2.5.2 Prediction Changes

Comparison	Changed	Rate
Part 1 vs. Global Only	24,472	20.4%
Part 1 vs. Hierarchical	24,362	20.3%
Global Only vs. Hierarchical	2,374	2.0%

Table 8: Prediction changes vs. Part 1 baseline (WeightedAvg strategy).

2.5.3 Kaggle Leaderboard Results

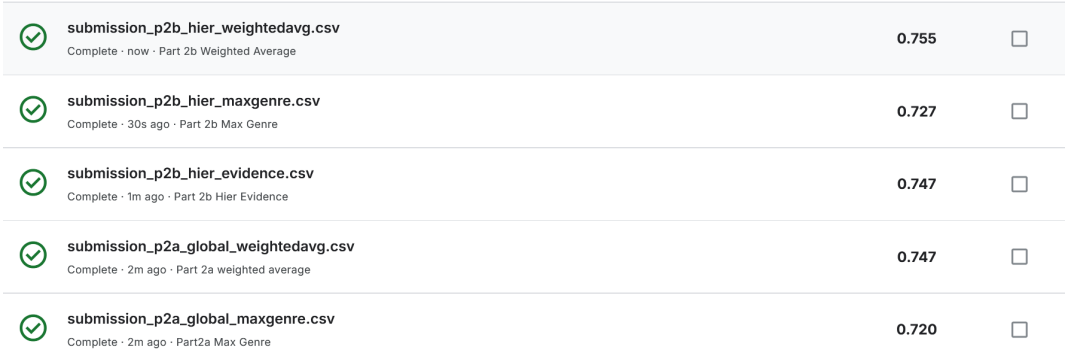

Figure 2: Kaggle AUC-ROC scores for the Part 2 strategies.

Part	Strategy	AUC-ROC	Δ vs. Part 1
1	S1: Max Genre Score	0.829	—
1	S2: Weighted Average	0.865	—
1	S3: Evidence-Weighted	**0.869**	—
2a	Global + MaxGenre	0.720	−0.109
2a	Global + WeightedAvg	0.747	−0.118
2b	Hier + MaxGenre	0.727	−0.102
2b	Hier + Evidence	0.747	−0.122
2b	Hier + WeightedAvg	0.755	−0.110

Table 9: Complete Kaggle AUC-ROC comparison: Part 1 vs. Part 2.

2.5.4 Key Findings — Why Fallback Scores Decreased

A critical and instructive result: the Part 2 cold-start strategies *decreased* AUC-ROC by 0.10–0.12 compared to Part 1. This is a well-documented phenomenon in recommender systems, and understanding *why* is the central takeaway.

1. **Global imputation introduces noise that drowns the personalized signal.** In Part 1, when a user has not rated an album, the album score is 0. This zero is not informative, but it is *harmless*: candidates with actual user ratings naturally dominate the ranking, while unrated candidates tie at zero and are ranked arbitrarily among themselves. The personalized signal stays clean.

   In Part 2, replacing those zeros with global averages (typically in the 50–80 range) gives *every* candidate a non-zero score. But these imputed scores are not personalized—they reflect crowd popularity, not individual taste. A track the user has never encountered can now outscore a track the user genuinely interacted with, purely because it is globally popular. This **dilutes the personalized signal** and degrades ranking quality.

2. **Absence-of-evidence ≠ evidence-of-absence.** Part 1's Evidence-Weighted strategy (S3, AUC = 0.869) already handled this correctly: it only counts a score when the user *actually* rated that hierarchy item, avoiding the "zero penalty" without introducing imputation noise. The Part 2 global fallback effectively undoes this advantage by treating "not rated" as "rated at the global average," which is a stronger and less accurate assumption.

3. **Dig Deeper provides a small relative lift within Part 2.** Comparing Part 2a (Global only) to Part 2b (Hierarchical), the Dig Deeper logic does improve scores slightly (e.g., WeightedAvg: 0.747 $\rightarrow$ 0.755, a +0.008 lift). This confirms that when sibling-track evidence exists, it is more predictive than the global average. However, the Dig Deeper rule only triggers for 4.7% of candidates, so its impact is limited.

4. **Data sparsity limits Dig Deeper's reach.** The Dig Deeper rule requires the user to have rated at least one other track in the same album—a rare condition in a dataset with 224,041 tracks spread across 31,141 albums. Only 5,678 of 120,000 candidates (4.7%) met this condition. This extreme sparsity is the fundamental constraint on the strategy's effectiveness.

5. **Lesson learned.** Naive feature imputation can be counterproductive in sparse recommendation settings. The optimal approach from Part 1 (Evidence-Weighted, AUC = 0.869) works precisely because it respects the distinction between "no data" and "low preference." A more sophisticated cold-start strategy would need to impute selectively—e.g., only for users with truly zero features—rather than applying global averages uniformly across all missing values.

3 All Submission Files

Part	Strategy	File	AUC
1	S1: Max Genre Score	`submission_strategy1_maxgenre.csv`	0.829
1	S2: Weighted Average	`submission_strategy2_weightedavg.csv`	0.865
1	S3: Evidence-Weighted	`submission_strategy3_evidence.csv`	**0.869**
2a	Global + MaxGenre	`submission_p2a_global_maxgenre.csv`	0.720
2a	Global + WeightedAvg	`submission_p2a_global_weightedavg.csv`	0.747
2b	Hier + MaxGenre	`submission_p2b_hier_maxgenre.csv`	0.727
2b	Hier + Evidence	`submission_p2b_hier_evidence.csv`	0.747
2b	Hier + WeightedAvg	`submission_p2b_hier_weightedavg.csv`	0.755

Table 10: Complete list of Kaggle submission files.

A Python Source Code

The complete Python implementation is contained in a single script (`EE627_Midterm.py`). The code is organized into data parsing, feature engineering, heuristic strategies, and the execution pipeline.

A.1 Data Parsing

```python
import os

SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(SCRIPT_DIR, "..", "Homework 4", "Data")
TRAIN_FILE = os.path.join(DATA_DIR, "trainItem2.txt")
TEST_FILE  = os.path.join(DATA_DIR, "testItem2.txt")
TRACK_FILE = os.path.join(DATA_DIR, "trackData2.txt")
ALBUM_FILE = os.path.join(DATA_DIR, "albumData2.txt")
OUTPUT_DIR = SCRIPT_DIR


def parse_training(path):
    """Parse training file into {user_id: {item_id: rating}}.
    Format: UserID|NumItems header, then ItemID<TAB>Rating lines.
    Items can be tracks, albums, artists, or genres."""
    users = {}
    with open(path, "r") as f:
        uid = None
        for line in f:
            line = line.strip()
            if "|" in line:
                uid = int(line.split("|")[0])
                users[uid] = {}
            elif "\t" in line and uid is not None:
                parts = line.split("\t")
                users[uid][int(parts[0])] = int(parts[1])
    return users


def parse_tracks(path):
    """Parse trackData2.txt -> {tid: (album, artist, [genres])}."""
    tracks = {}
    with open(path, "r") as f:
        for line in f:
            parts = line.strip().split("|")
            if len(parts) < 3:
                continue
            tid = int(parts[0])
```

```python
            alb = None if parts[1] == "None" else int(parts[1])
            art = None if parts[2] == "None" else int(parts[2])
            genres = [int(g) for g in parts[3:]
                      if g.strip() and g != "None"]
            tracks[tid] = (alb, art, genres)
    return tracks


def parse_test(path):
    """Parse testItem2.txt -> [(user_id, [track_ids])]."""
    test = []
    with open(path, "r") as f:
        uid, tids = None, []
        for line in f:
            line = line.strip()
            if "|" in line:
                if uid is not None:
                    test.append((uid, tids))
                uid = int(line.split("|")[0])
                tids = []
            elif line:
                tids.append(int(line))
        if uid is not None:
            test.append((uid, tids))
    return test
```

Listing 1: Data parsing functions.

A.2 Feature Engineering (Part 1)

```python
def compute_feature_vector(user_ratings, track_meta, uid, tid):
    """Build feature vector for one (user, track) pair."""
    ratings = user_ratings.get(uid, {})
    alb_id, art_id, genre_ids = track_meta.get(
        tid, (None, None, []))

    # Album score
    if alb_id is not None and alb_id in ratings:
        album_score, has_album = ratings[alb_id], 1
    else:
        album_score, has_album = 0, 0

    # Artist score
    if art_id is not None and art_id in ratings:
        artist_score, has_artist = ratings[art_id], 1
    else:
        artist_score, has_artist = 0, 0

    # Genre statistics
    genre_scores = [ratings[g] for g in genre_ids
                    if g in ratings]
    n = len(genre_scores)
    if n > 0:
        g_max  = max(genre_scores)
        g_min  = min(genre_scores)
        g_mean = sum(genre_scores) / n
        g_var  = sum((s - g_mean)**2 for s in genre_scores) / n
        g_sum  = sum(genre_scores)
    else:
        g_max = g_min = g_sum = 0
        g_mean = g_var = 0.0

    return {
        "album_score": album_score,
        "artist_score": artist_score,
        "has_album": has_album,    "has_artist": has_artist,
        "genre_count": n,          "genre_max": g_max,
        "genre_min": g_min,        "genre_mean": g_mean,
        "genre_var": g_var,        "genre_sum": g_sum,
    }
```

Listing 2: Feature engineering (Part 1.a).

A.3 Heuristic Strategies (Part 1)

```python
def strategy_max_genre(features_list):
    """Strategy 1: 0.30*album + 0.20*artist + 0.50*genre_max"""
    return [0.30 * f["album_score"]
            + 0.20 * f["artist_score"]
            + 0.50 * f["genre_max"]
            for f in features_list]


def strategy_weighted_avg(features_list):
    """Strategy 2: 0.40*album + 0.35*artist + 0.25*genre_mean"""
    return [0.40 * f["album_score"]
            + 0.35 * f["artist_score"]
            + 0.25 * f["genre_mean"]
            for f in features_list]


def strategy_evidence_weighted(features_list):
    """Strategy 3: has_album*album + has_artist*artist
       + genre_count*genre_mean/10"""
    return [f["has_album"] * f["album_score"]
            + f["has_artist"] * f["artist_score"]
            + f["genre_count"] * f["genre_mean"] / 10.0
            for f in features_list]
```

Listing 3: Heuristic strategies (Part 1.b).

A.4 Ranking and Submission Pipeline

```python
def rank_top3(scores, track_ids):
    """Pick top-3 scored tracks as '1', rest as '0'."""
    paired = sorted(zip(scores, track_ids),
                    key=lambda x: x[0], reverse=True)
    top3 = set(tid for _, tid in paired[:3])
    return {tid: (1 if tid in top3 else 0)
            for tid in track_ids}


def write_submission(results, path):
    with open(path, "w") as f:
        f.write("TrackID,Predictor\n")
        for key, label in results:
            f.write(f"{key},{label}\n")


def run_strategy(user_ratings, track_meta, test_users,
                 strategy_fn, out_path):
    results = []
    for uid, candidates in test_users:
        feats  = [compute_feature_vector(
                      user_ratings, track_meta, uid, tid)
                  for tid in candidates]
        scores = strategy_fn(feats)
        recs   = rank_top3(scores, candidates)
        for tid in candidates:
            results.append((f"{uid}_{tid}", recs[tid]))
    write_submission(results, out_path)
    return results
```

Listing 4: Ranking and submission pipeline.

A.5 Main Entry Point (Part 1)

```python
if __name__ == "__main__":
    user_ratings = parse_training(TRAIN_FILE)
    track_meta   = parse_tracks(TRACK_FILE)
    test_users   = parse_test(TEST_FILE)

    # Strategy 1: Max Genre Score
    run_strategy(user_ratings, track_meta, test_users,
        strategy_max_genre,
        os.path.join(OUTPUT_DIR,
            "submission_strategy1_maxgenre.csv"))

    # Strategy 2: Weighted Average
    run_strategy(user_ratings, track_meta, test_users,
        strategy_weighted_avg,
        os.path.join(OUTPUT_DIR,
            "submission_strategy2_weightedavg.csv"))

    # Strategy 3: Evidence-Weighted
    run_strategy(user_ratings, track_meta, test_users,
        strategy_evidence_weighted,
        os.path.join(OUTPUT_DIR,
            "submission_strategy3_evidence.csv"))
```

Listing 5: Main entry point.

A.6 Part 2: Cold-Start Extensions

```python
from collections import defaultdict

def build_album_to_tracks(track_meta):
    """Build {album_id: [track_ids]} for Dig Deeper."""
    album_tracks = defaultdict(list)
    for tid, (alb, _, _) in track_meta.items():
        if alb is not None:
            album_tracks[alb].append(tid)
    return dict(album_tracks)


def build_global_stats(user_ratings):
    """Global mean rating per item across all users.
    Returns {item_id: (mean_rating, num_raters)}."""
    item_sum = defaultdict(float)
    item_cnt = defaultdict(int)
    for _, ratings in user_ratings.items():
        for item_id, rating in ratings.items():
            item_sum[item_id] += rating
            item_cnt[item_id] += 1
    return {iid: (item_sum[iid] / item_cnt[iid],
                  item_cnt[iid])
            for iid in item_sum}
```

Listing 6: Album-to-tracks index and global statistics (Part 2 setup).

```python
def compute_feature_vector_v2(user_ratings, track_meta,
        uid, tid, album_to_tracks, global_stats,
        use_dig_deeper=True, use_global=True):
    ratings = user_ratings.get(uid, {})
    alb_id, art_id, genre_ids = track_meta.get(
        tid, (None, None, []))

    # Album: Direct -> Dig Deeper -> Global
    album_score, has_album, album_source = 0, 0, "none"
    if alb_id is not None:
        if alb_id in ratings:
            album_score = ratings[alb_id]
            has_album, album_source = 1, "direct"
        elif use_dig_deeper:
            siblings = album_to_tracks.get(alb_id, [])
            sib_ratings = [ratings[st] for st in siblings
                           if st in ratings and st != tid]
            if sib_ratings:
                album_score = sum(sib_ratings)/len(sib_ratings)
                has_album, album_source = 1, "dig_deeper"
            elif use_global and alb_id in global_stats:
                album_score = global_stats[alb_id][0]
                has_album, album_source = 1, "global"
        elif use_global and alb_id in global_stats:
            album_score = global_stats[alb_id][0]
            has_album, album_source = 1, "global"

    # Artist: Direct -> Global
    artist_score, has_artist = 0, 0
    if art_id is not None:
        if art_id in ratings:
            artist_score, has_artist = ratings[art_id], 1
        elif use_global and art_id in global_stats:
            artist_score = global_stats[art_id][0]
            has_artist = 1

    # Genre: Direct -> Global (per genre)
    genre_scores = []
    for gid in genre_ids:
        if gid in ratings:
            genre_scores.append(ratings[gid])
        elif use_global and gid in global_stats:
            genre_scores.append(global_stats[gid][0])
    # ... compute genre stats as in Part 1 ...
```

Listing 7: Enhanced feature vector with hierarchical fallback.